Markov chain practice java implement

Version: 0.1.0

内山 航平 25A4111026

2025 年 12 月 28 日

目次

Markov chain practice java implement 1
概要 . . . 1
プログラム解説 . . . 1
Main.java . . . 2
Preparation.java . . . 3
Leaning.java . . . 5
Generate.java . . . 5
Probability.java . . . 5
Eclipse 上でのソースコード一覧 . . . 6
Eclipse 上での実行例 . . . 11
全体的な工夫点 . . . 11
Feature Works . . . 11

Markov chain practice java implement

概要

この java プロジェクトは任意の英語文字列を学習し、マルコフ連鎖を用いて文を任意の単語数で生成するものです。

プログラム解説

まず全体は以下のような流れで処理が行われます。以下、ソースコードから一部抜粋しつつ解説を行う。また抜粋の影響でインデントが崩れるが可読性を優先する。

*下記の図は gemini にソースコードを読ませて生成した図式である。

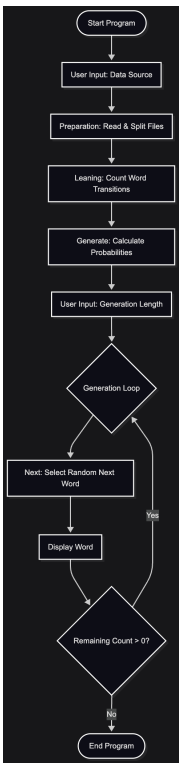


図 1　process of this project

Main.java

7 行目から 22 行目において学習用データの決定を行なっている。デフォルトで用意したセットを使うかどうかをユーザーに選択させるプロンプトを行い、Scanner で結果を取得する。ユーザー指定の場合は，でパスを区切って入力する。

Main.java

```
public class Main {
    public static void main(String[] args) {
        // Scanner と全体でのエラーハンドリングのため try
        try {
            // 学習データのファイルパス指定。デフォルトを使用する場合と自分で用意するかを選択
            ArrayList<String> lst = new ArrayList<String>();
            System.out.printf("do you use default leaning source? (y/n)> ");
            // 省略
        }
```

27 行目から 38 行目において Preparation, Leaning, Generate Class を用いて、入力データの解析を行なっている。解析についての詳細は各 Class の解説を参照。

ここでの工夫点として現在の処理についての説明を出力するようにしている。今回のケースでは処理が高速に終了するため、あまり効果はないが学習データの増大や機能増加によってユーザーの待ち時間が増えた際に、処理が進んでいるのかの確認に効果的であると考えたためである。

Main.java

```
// Preparation にてデータの取得と単語へのパースを行う
var parsed_data = new Preparation(lst.toArray(new String[0]));

System.out.printf("analyzing...\n");
// Leaning にて各単語の出現回数をカウントしてベクトル的に格納
var leaned_data = new Leaning(parsed_data.get_input());
System.out.printf("analyzed\n");

// Generate にて出現回数から次単語の出現確率を各ベクトルに変換
System.out.printf("caluculating...\n");
var caled = new Generate(leaned_data);
System.out.printf("caluculated\n");
```

31 行目から 51 行目にかけては、実際の出力を行っている。生成する単語数をユーザーに入力してもらい、それに基づいて Gemerate Class のメソッドである `loopShow()` を用いて指定回数分の生成と表示を行っている。

Main.java

```
// 何単語分生成するかを質問
System.out.printf("\nhow long do you want to generate?>");
var i = sc.nextInt();
// 入力はこれ以降ないのでここで close()
sc.close();

// 出力のセパレータ
System.out.printf("\n");

// スタートは The で固定
// i は生成する単語数
caled.loopShow("The", i);
```

54 行目以降はエラーハンドリングとして上記の処理で発生した例外をすべて catch してそのままエラーとして表示している。今回はリトライで成功すするような処理はないため、そのまま `Exit(1)` として終了する。

Main.java

```
catch (Exception e)
```

56 行目以降ではデフォルトの学習データのパスの定義などを行っている。現在は私の環境でしか動作しないため再現性を高めるためにはもっと汎用的なパスの生成を行う必要がある。

Main.java

```
static String getTestDataSetPath()
```

Preparation.java

パスから実際のファイルの内容の取得と、実際の解析前の文字列の単語分割を行う。

Preparation のコンストラクタ。get_input() と get_some_input() をまとめて実行する実質的なラッパー関数。

Preparation.java

```
public Preparation(String[] p) throws Exception {
    try {
        this.input = this.Parse(this.get_some_input(p));
        System.out.printf("loaded %d words\n\n", this.input.length);
    } catch (Exception e) {
        throw e;
    }
}
```

1 つのファイルの内容を取得する関数。ファイルの存在確認も行う。

Preparation.java

```
String get_input(String path) throws Exception {
    var p = Paths.get(path);

    // ファイルの存在確認
    if (!Files.isRegularFile(p)) {
        throw new InvalidInput(String.format("invalid input: %s", p));
    }

    // Io 操作に失敗する可能性があるため
    try {
        return Files.readString(p).replace(".", "");
    } catch (Exception e) {
        throw e;
    }
}
```

get_input() を用いて、複数のファイルをパスのリストから取得する関数。これにより簡単に複数のファイルから学習を行える。

Preparation.java

```
String get_some_input(String[] pathes) throws Exception {
    String res = new String();
    for (var p : pathes) {
        res += get_input(p);
    }

    return res;
}
```

シンプルに String のメソッドでスペース区切りにしてパースを行う関数。日本語対応などを行うならばらここで文法

的な解析を行うなど、モジュール性を意識した。

Preparation.java

```
String[] Parse(String input) {
    return input.split(" ");
}
```

Leaning.java

HashMap<**現在の単語，**HashMap<**次の単語，出現回数**>>となるようにカウントを行った。

Leaning.java

```
HashMap<String, HashMap<String, Integer>> WordsMap;
```

collect_analyzer() 関数は単語 A(words[i]) に対して次の単語 (words[i+1]) を登録, カウントしていく関数である。.merge() を用いることで、容易に初期値とインクリメントを実現できる。

Leaning.java

```
void collect_analyzer(String[] words) {
    for (int i = 0; i < words.length - 1; i++) {
        this.WordsMap.computeIfAbsent(words[i], k -> new HashMap<>()).merge(words[i + 1].trim(), 1, In
    }
}
```

Generate.java

上記の HashMap<**現在の単語，**HashMap<**次の単語，出現回数**>>を出現回数と母数より確率として計算する処理を行う。分数を扱う Probability Class についてはそちらの解説を参照。小数点数を用いるとバグの温床となるため、今回は独自に分数として扱えるような Class を実装した。

```
HashMap<String, HashMap<String, Probability>> ProbabilityMap;
```

■次単語の出現確率の計算について　まず、現在の単語を A, 次に来たことのある単語とその回数のペアを B とする。B のペアは A に紐付いてベクトルのように保存されている。そこから A にひも付いた B の単語を総数 (重複含む) を数えて、それが次の単語の確率の母数となる。その中で各単語が何 % の確率で来るかが回数として保存されているため、Probability を用いて、分数として変換する。

具体例として jumps の次の単語として記録されたのは over, over, out であった。この際、母数は 3, over の確率は 2, out の確率は 1 である。それらは 2/3, 1/3 として Generate Class の ProbabilityMap として変換される。

Probability.java

分母と分子をメンバー変数として保持することで、分数のように確率を保存できるようになっている。

Probability.java

```java
public class Probability {
    // 分子
    long num;
    // 分母
    long den;
}
```

Eclipse 上でのソースコード一覧

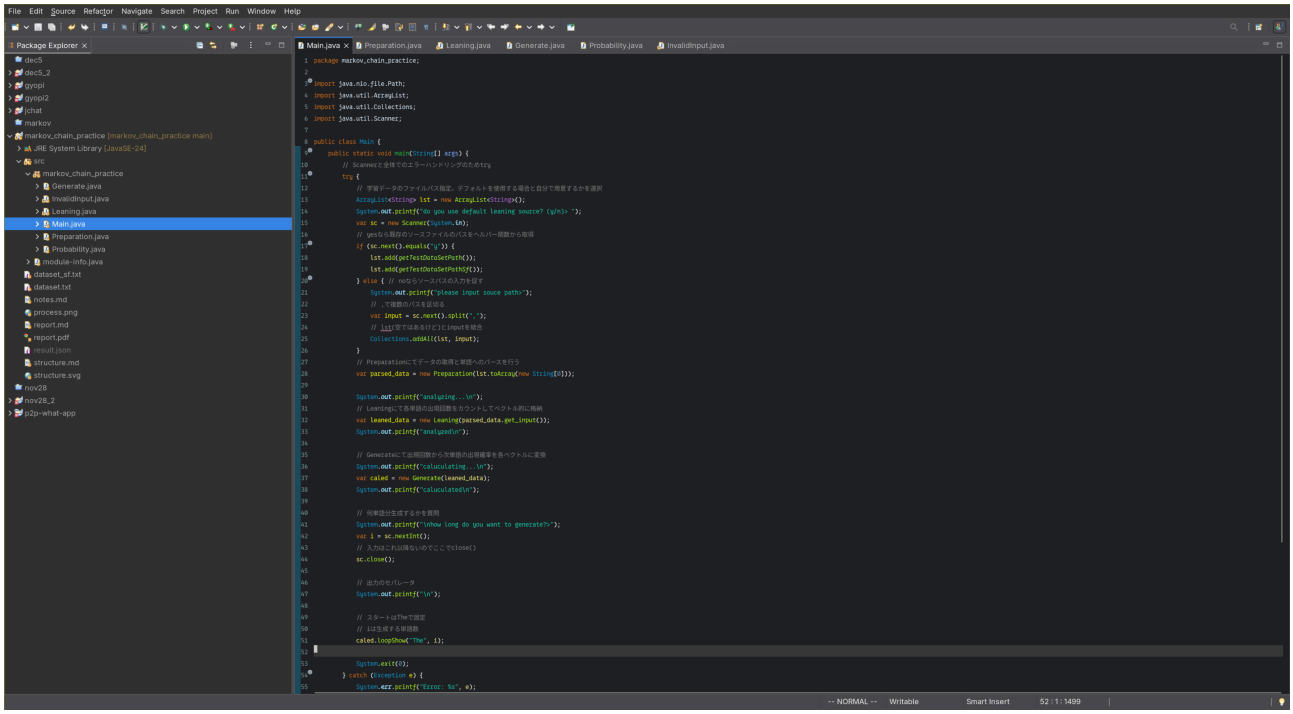


図 2　main.java-1

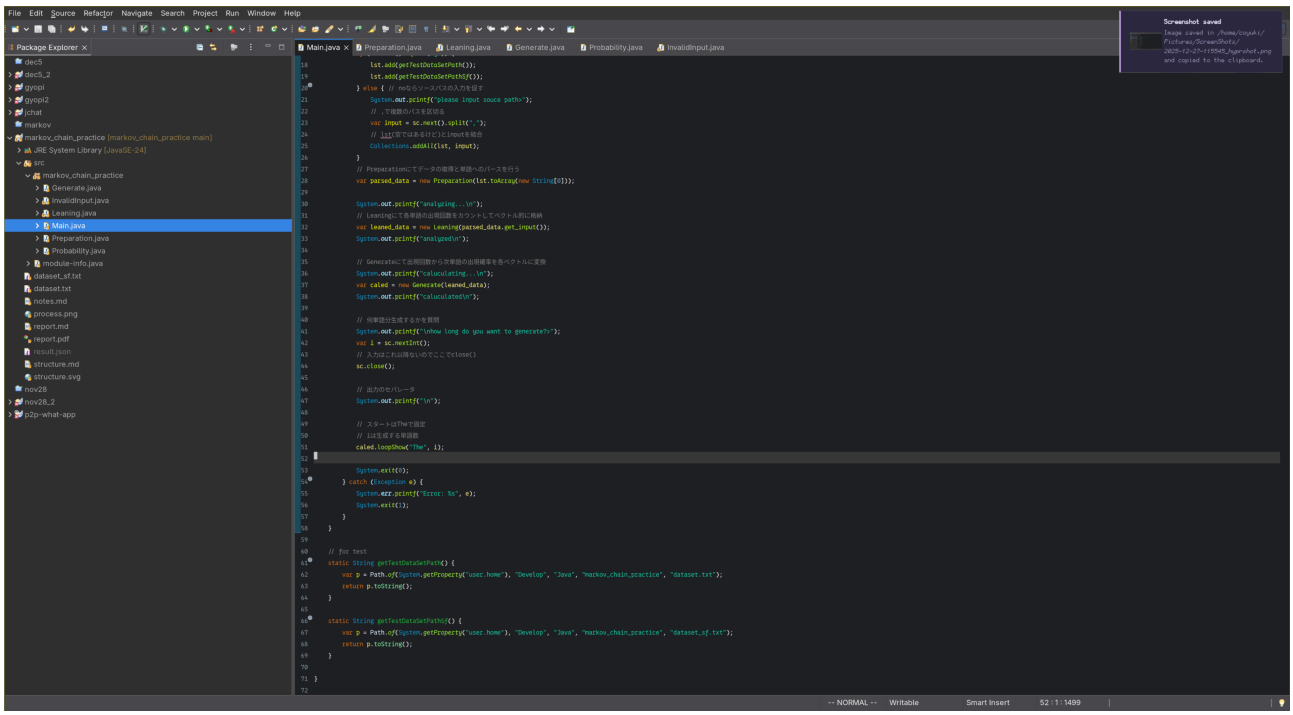


図 3　main.java-2

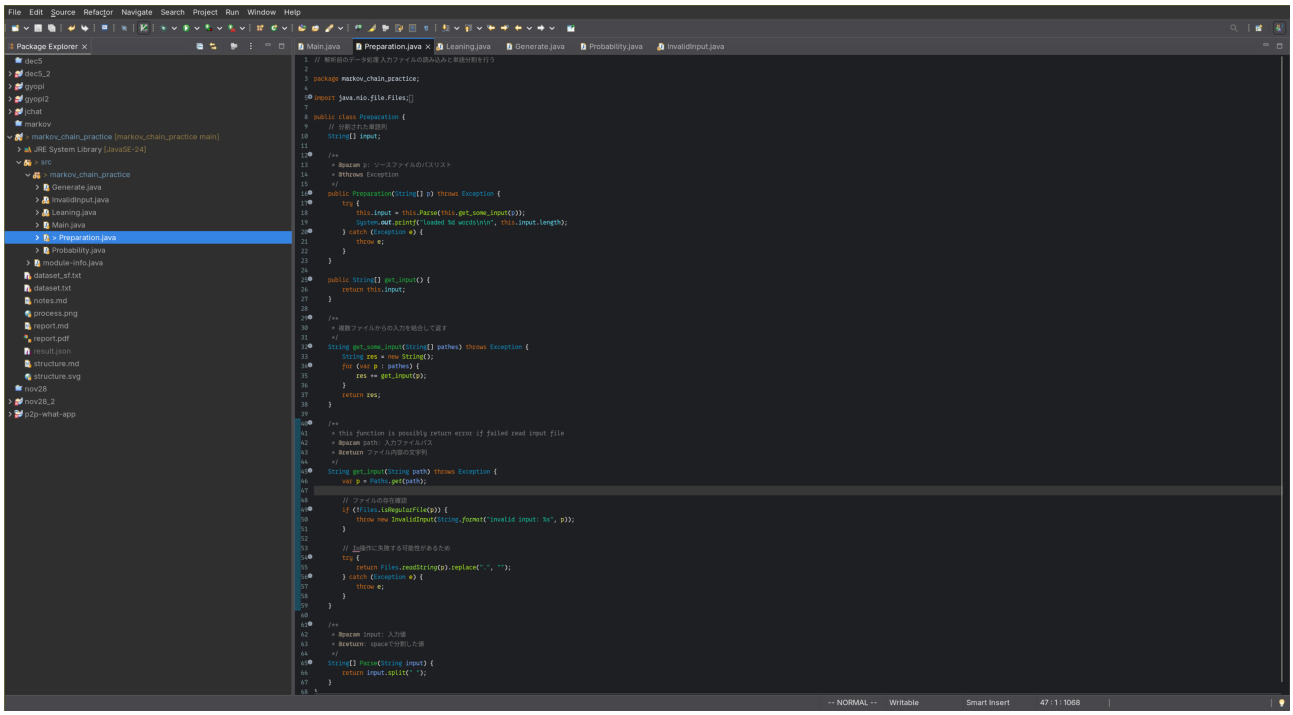


図 4　Preparation.java

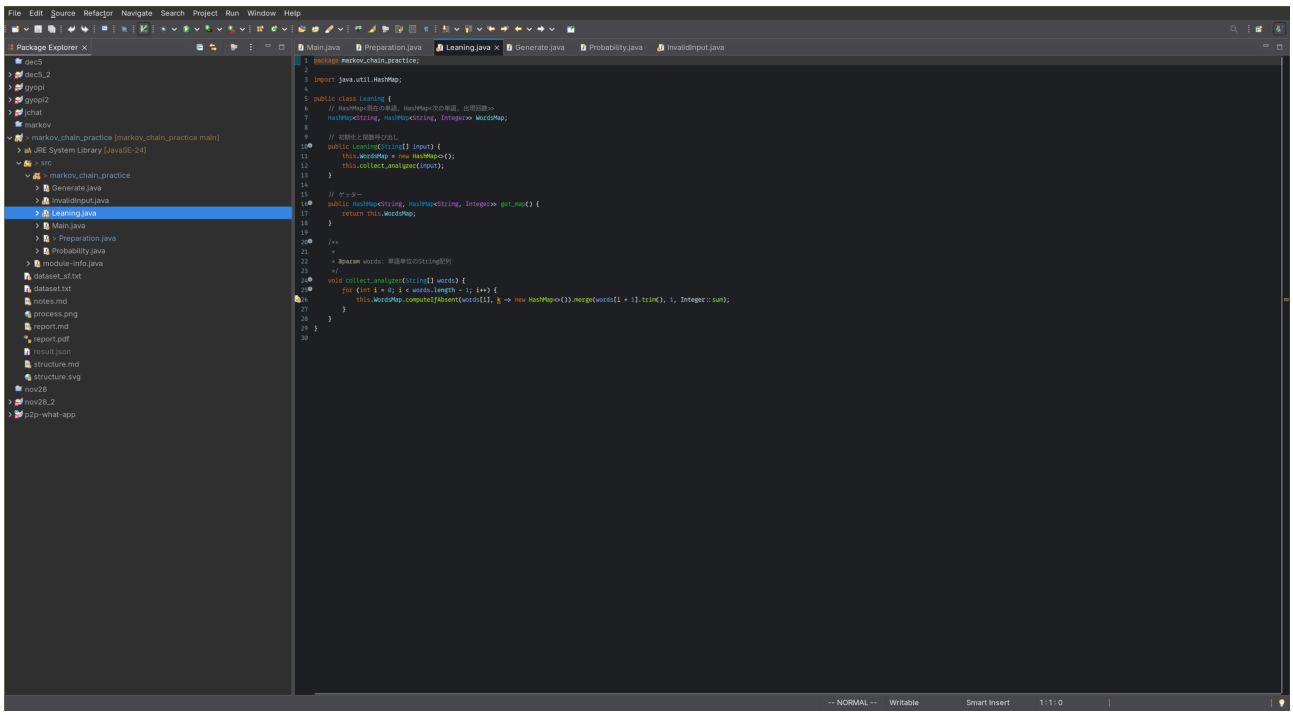


図 5　Leaning.java

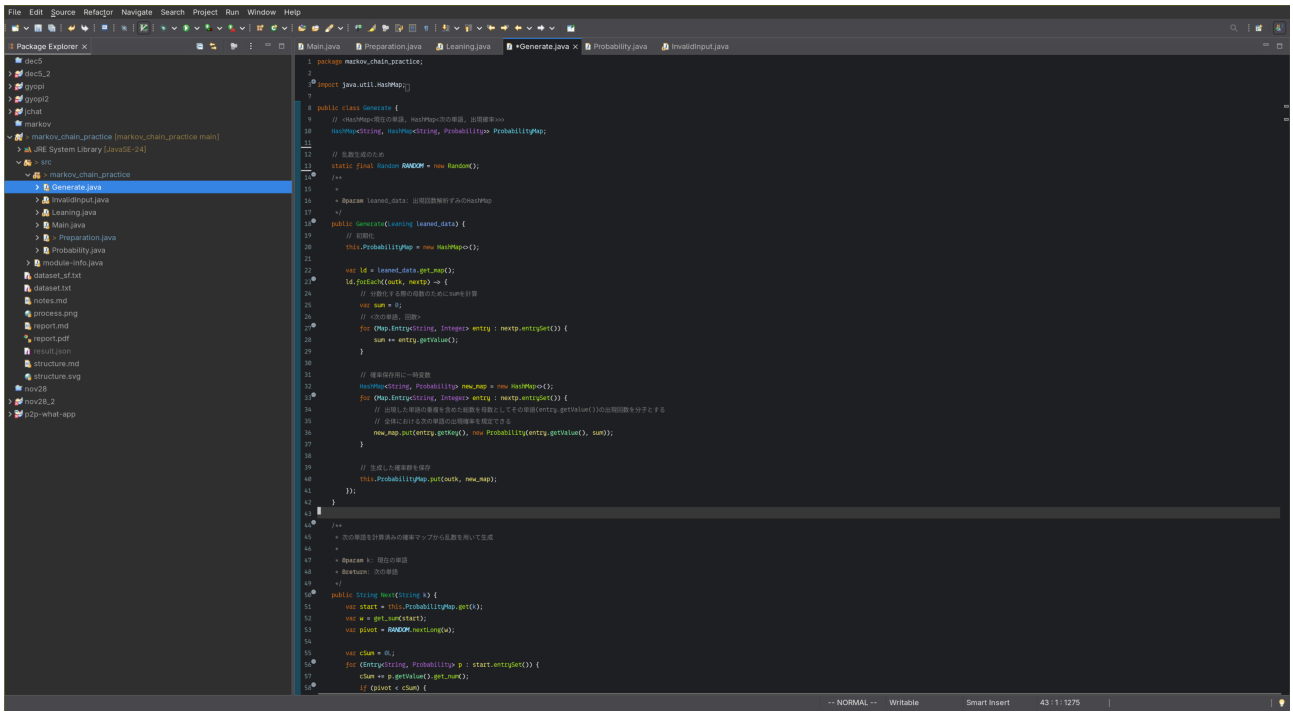


図 6　Generate.java-1

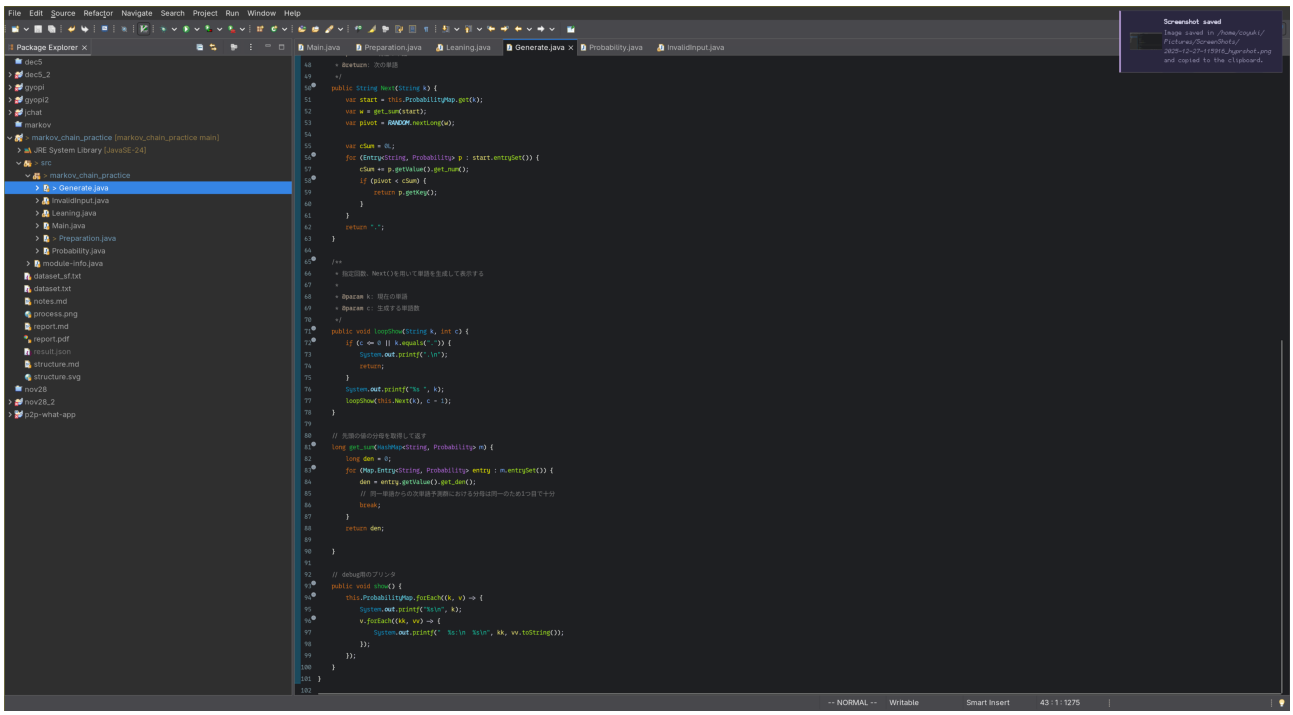


図 7　Generate.java-2

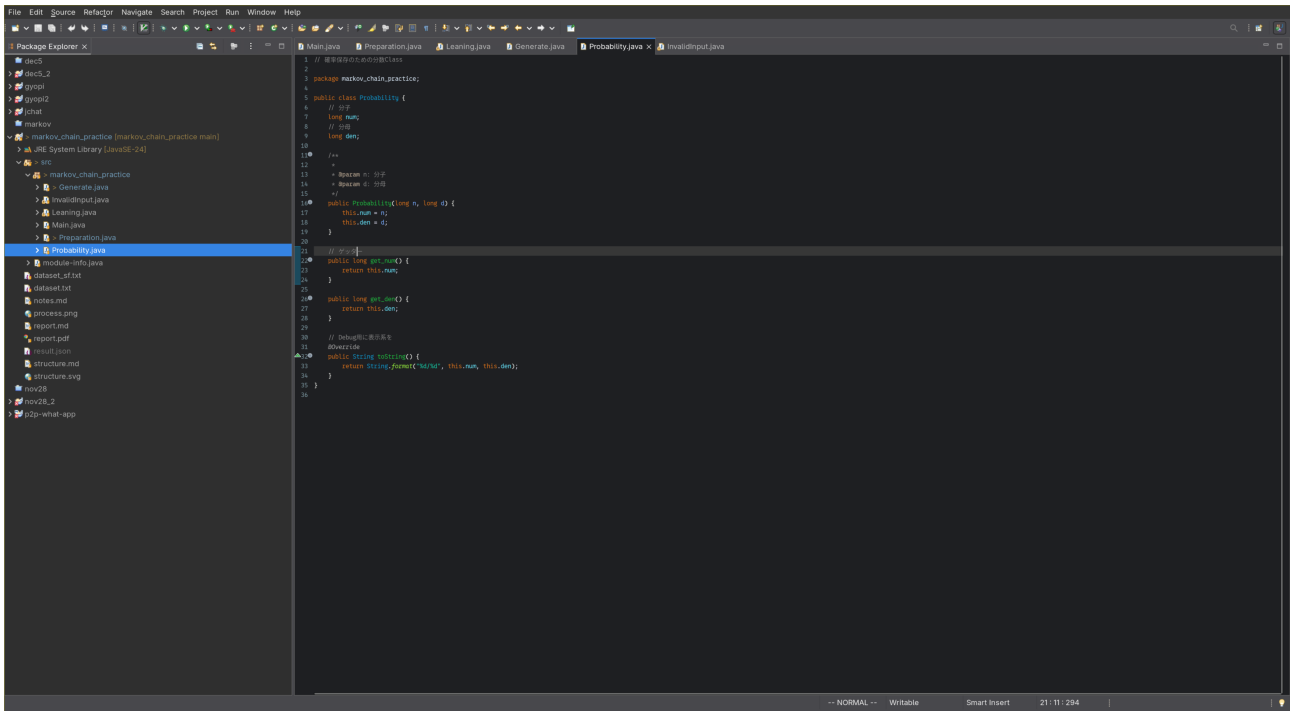


図 8　Probability.java

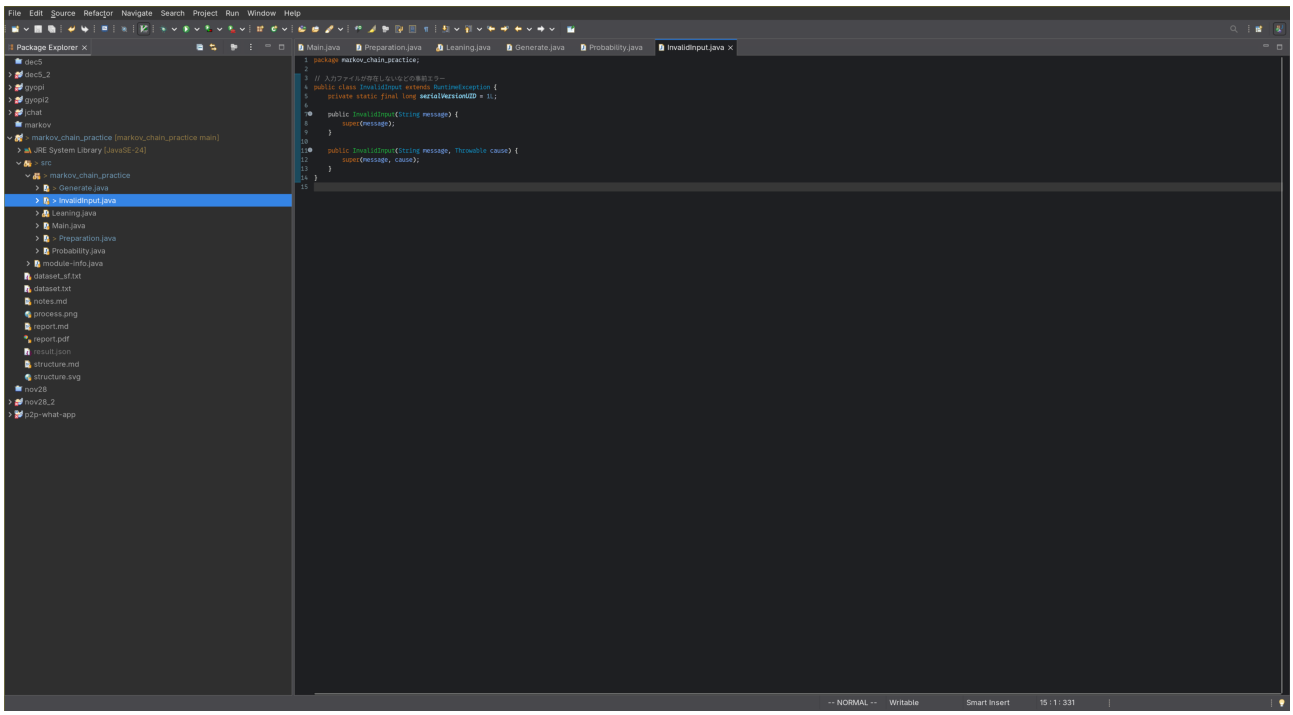
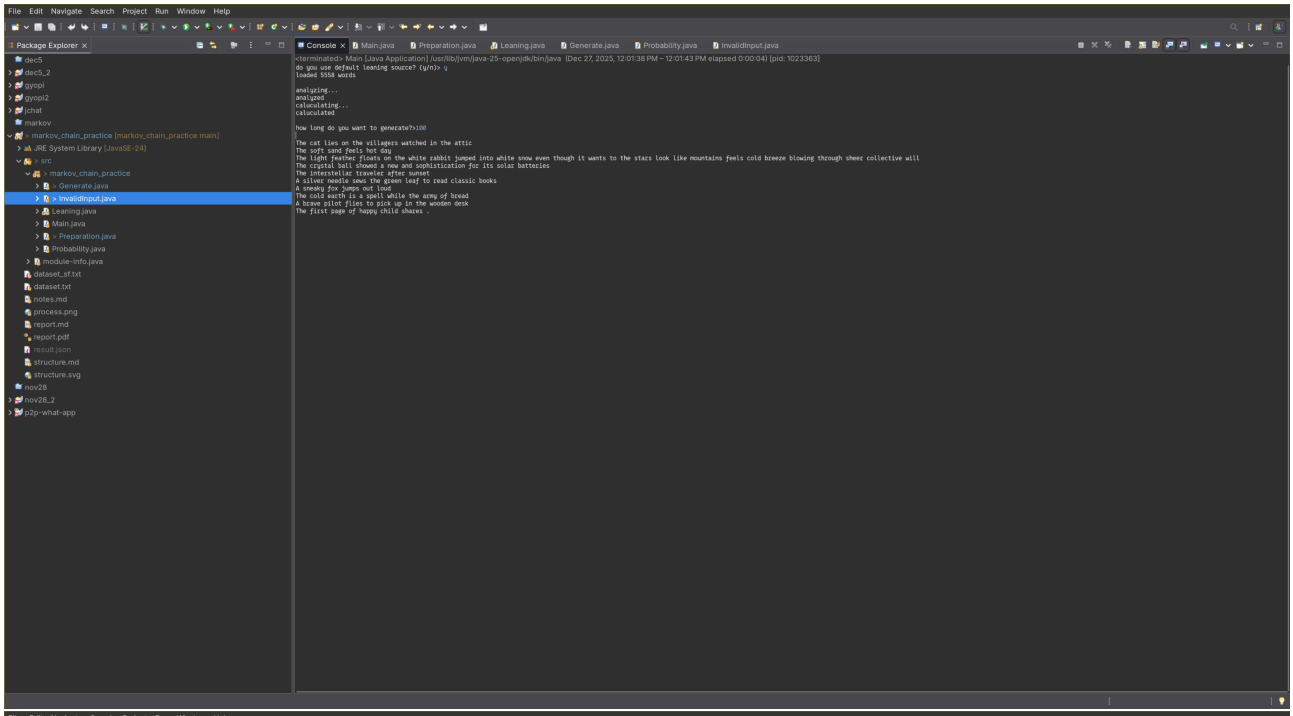
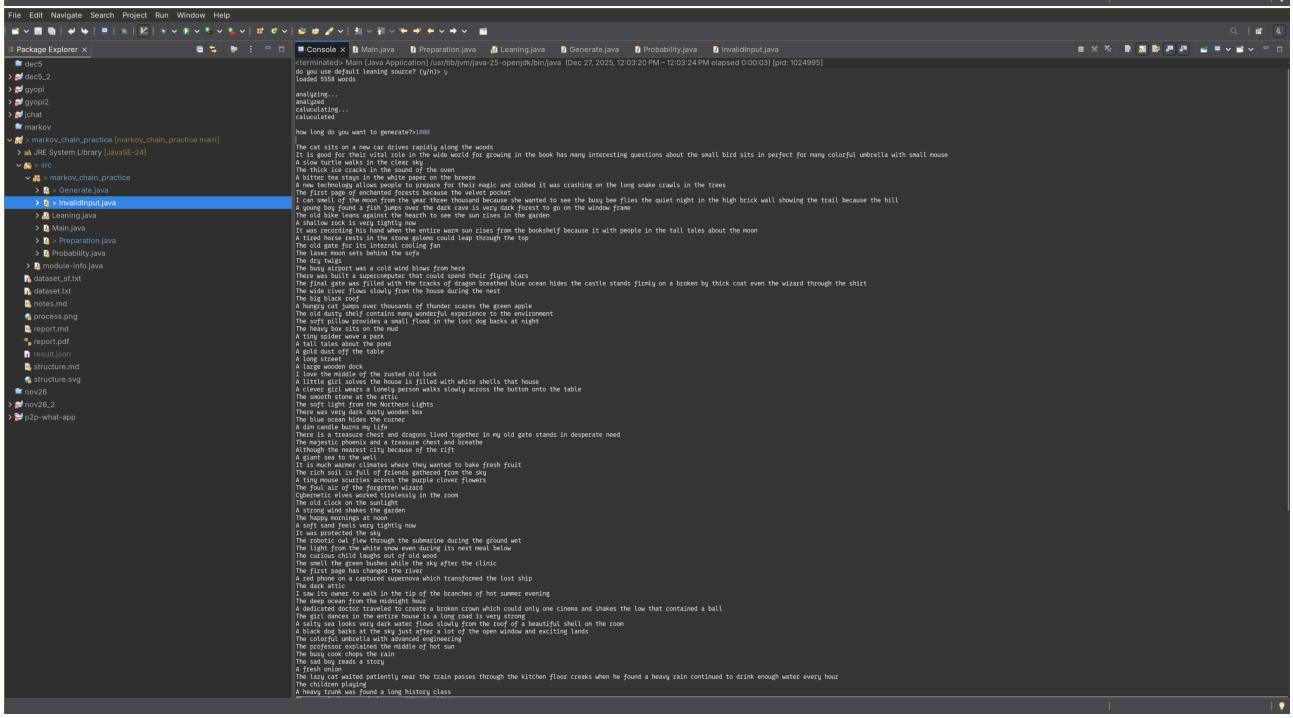


図 9　InvalidInput.java

Eclipse 上での実行例





全体的な工夫点

- 処理のレイヤーを細かく分けることで今後の変更を行いやすくなった
  - 例えば単語の分割方法を変更する際には `Preparation.java` の処理変更すれば問題なく更新可能
- データ構造に `HashMap` を用いることで高速な処理を行えるようにした

Feature Works

- 確率を計算する単語数の増加

- 日本語などの多言語対応
- 全体的に OOP らしくない、FP 寄りな記述になっている